

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA0304

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 40%

Dr. Dumpala Prasanth

QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Code of Conduct:

I Y.THANUSHMA(192425355) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Y.Thanushma

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

EXPERIMENT 1: INSTALLATION OF REINFORCEMENT LEARNING DEVELOPMENT ENVIRONMENT

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program

Program:

```
import tensorflow as tf
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version :", tf.__version__)
print("Gymnasium Imported Successfully")
print("NumPy Version :", np.__version__)
print("Matplotlib Imported Successfully")

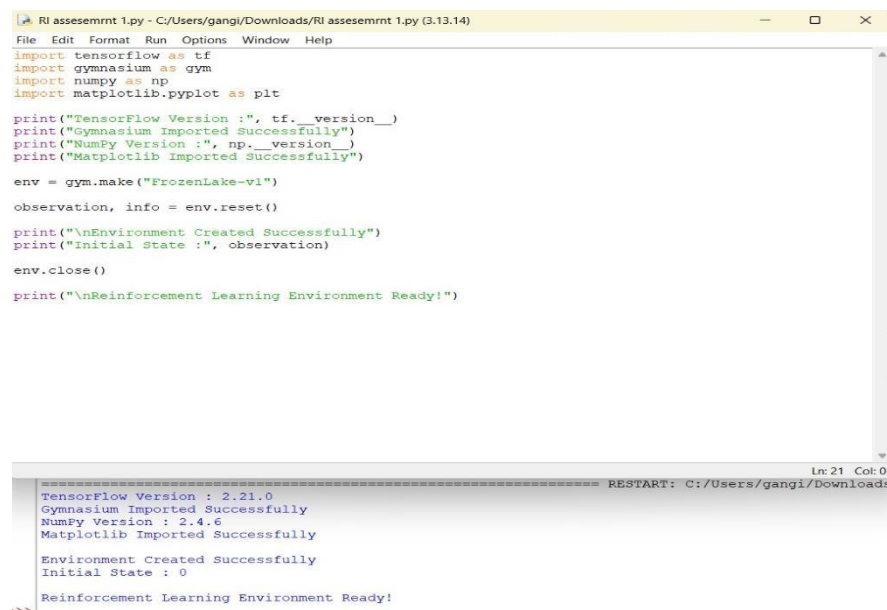
env = gym.make("FrozenLake-v1")
observation, info = env.reset()

print("\nEnvironment Created Successfully")
print("Initial State :", observation)

env.close()

print("\nReinforcement Learning Environment Ready!")
```

Output:



The screenshot shows a Python script execution window titled "RI assesemrnt 1.py - C:/Users/gangli/Downloads/RI assesemrnt 1.py (3.13.14)". The script content is the same as the one in the "Program" section. The output window at the bottom shows the following results:

```
TensorFlow Version : 2.21.0
Gymnasium Imported Successfully
NumPy Version : 2.4.6
Matplotlib Imported Successfully

Environment Created Successfully
Initial State : 0

Reinforcement Learning Environment Ready!
>>>
```

EXPERIMENT 2: FAMILIARIZATION WITH OPENAI GYM/GYMNASIUM ENVIRONMENTS

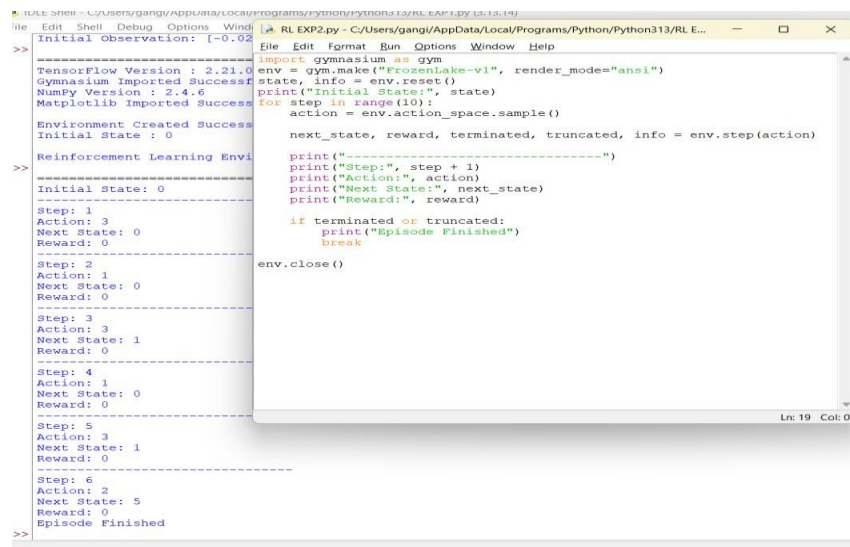
AIM:

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and Mountain Car using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

PROGRAM:

```
import gymnasium as gym
env = gym.make("FrozenLake-v1", render_mode="ansi")
state, info = env.reset()
for step in range(10):
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)
    print("-----")
    print("Step:", step + 1)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)
    if terminated or truncated:
        print("Episode Finished")
    break
env.close()
```

Output:



The screenshot shows a Jupyter Notebook on the left and a terminal window on the right. The Jupyter Notebook displays the output of the program, showing the initial state, action, next state, reward, and episode finished message. The terminal window shows the code being executed, which is the same code as in the 'PROGRAM' section. The terminal output matches the Jupyter Notebook output.

```
Initial Observation: [-0.02]
>>>
-----
TensorFlow Version : 2.21.0
Gymnasium Imported Successfully
NumPy Version : 2.4.6
Matplotlib Imported Successfully
Environment Created Successfully
Initial State : 0
Reinforcement Learning Environment
>>>
Initial State: 0
-----
Step: 1
Action: 3
Next State: 0
Reward: 0
-----
Step: 2
Action: 1
Next State: 0
Reward: 0
-----
Step: 3
Action: 3
Next State: 1
Reward: 0
-----
Step: 4
Action: 1
Next State: 0
Reward: 0
-----
Step: 5
Action: 3
Next State: 1
Reward: 0
-----
Step: 6
Action: 2
Next State: 5
Reward: 0
Episode Finished
>>>
```

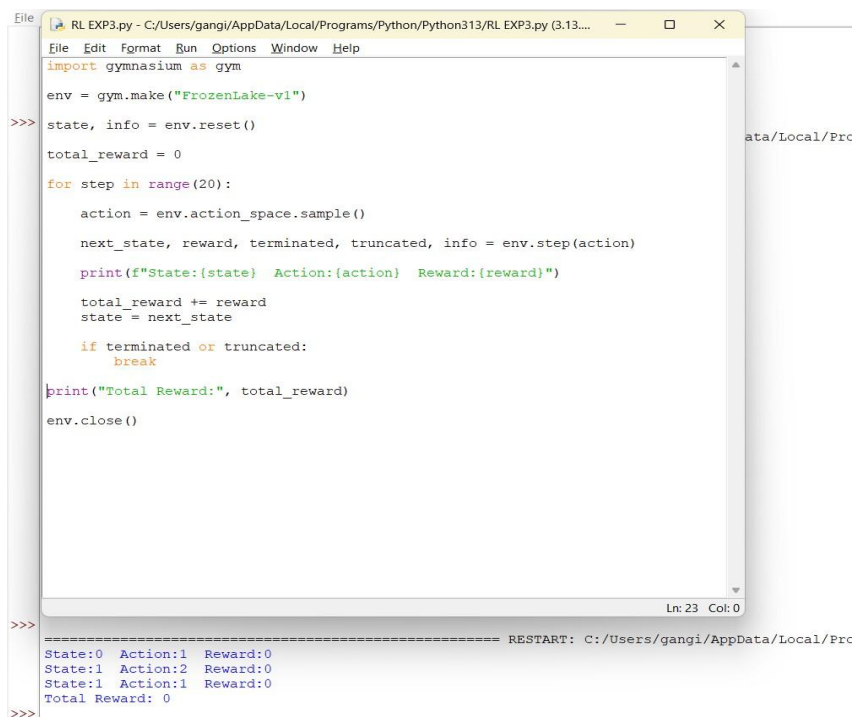
EXPERIMENT 3: IMPLEMENTATION OF THE REINFORCEMENT LEARNING FRAMEWORK

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

PROGRAM:

```
import gymnasium as gym env =
gym.make("FrozenLake-v1")
state, info = env.reset() total_reward = 0 for step in range(20):
action = env.action_space.sample() next_state, reward,
terminated, truncated, info = env.step(action)
print(f"State:{state} Action:{action} Reward:{reward}")
total_reward += reward state = next_state if terminated or
truncated:
break
print("Total Reward:", total_reward)
env.close()
```

OUTPUT



```
File RL EXP3.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP3.py (3.13...
File Edit Format Run Options Window Help
import gymnasium as gym
env = gym.make("FrozenLake-v1")
>>> state, info = env.reset()
total_reward = 0
for step in range(20):
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)
    print(f"State:{state} Action:{action} Reward:{reward}")
    total_reward += reward
    state = next_state
    if terminated or truncated:
        break
print("Total Reward:", total_reward)
env.close()
Ln: 23 Col: 0
>>> ===== RESTART: C:/Users/gangi/AppData/Local/Pr
State:0 Action:1 Reward:0
State:1 Action:2 Reward:0
State:1 Action:1 Reward:0
Total Reward: 0
>>>
```

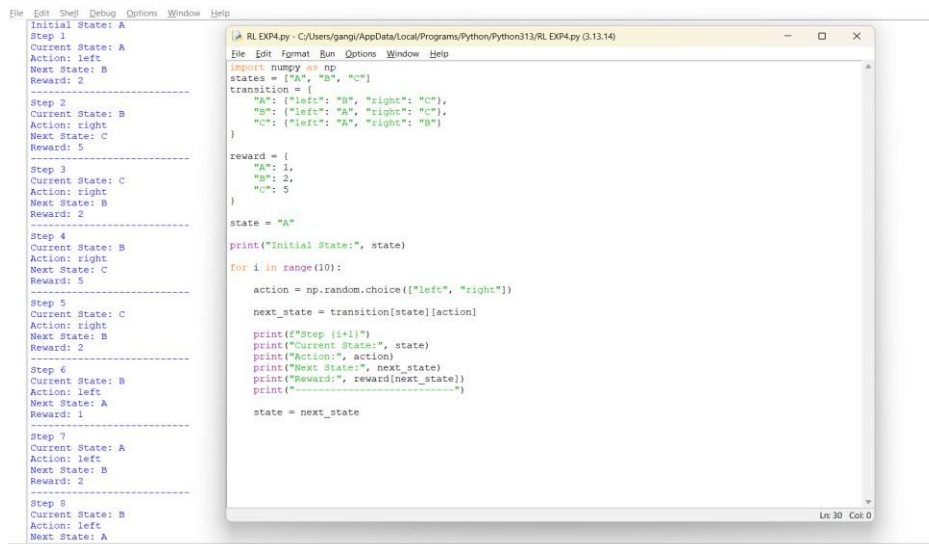
EXPERIMENT 4: SIMULATION OF A MARKOV DECISION PROCESS (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems

PROGRAM:

```
import numpy as np
states = ["A", "B",
"C"] transition = {
    "A": {"left": "B", "right": "C"},
    "B": {"left": "A", "right": "C"},
    "C": {"left": "A", "right": "B"}
} reward
= {
    "A": 1,
    "B": 2,
    "C": 5 }
state =
"A"
print("Initial State:", state) for i in range(10):
action = np.random.choice(["left", "right"])
next_state = transition[state][action]
print(f"Step {i+1}")    print("Current State:",
state)    print("Action:", action)
print("Next State:", next_state)
print("Reward:", reward[next_state])
print("-----")    state =
next_state
```

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window displays the output of a reinforcement learning experiment, showing the sequence of states, actions, and rewards over 10 steps. The right window shows the Python code that generates this output.

```
Initial State: A
Step 1
Current State: A
Action: left
Next State: B
Reward: 2
-----
Step 2
Current State: B
Action: right
Next State: C
Reward: 5
-----
Step 3
Current State: C
Action: right
Next State: B
Reward: 2
-----
Step 4
Current State: B
Action: right
Next State: C
Reward: 5
-----
Step 5
Current State: C
Action: right
Next State: B
Reward: 2
-----
Step 6
Current State: B
Action: left
Next State: A
Reward: 1
-----
Step 7
Current State: A
Action: left
Next State: B
Reward: 2
-----
Step 8
Current State: B
Action: left
Next State: A
```

```
RL EXP4.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP4.py (3.13.14)
import numpy as np
states = ["A", "B", "C"]
transition = {
    "A": {"left": "B", "right": "C"},
    "B": {"left": "A", "right": "C"},
    "C": {"left": "A", "right": "B"}
}
reward = {
    "A": 1,
    "B": 2,
    "C": 5
}
state = "A"
print("Initial State:", state)
for i in range(10):
    action = np.random.choice(["left", "right"])
    next_state = transition[state][action]
    print(f"Step {i+1}")
    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward[next_state])
    print("-----")
    state = next_state
```

EXPERIMENT 5: BELLMAN EQUATION DEMONSTRATION

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

PROGRAM:

```
import numpy as np
gamma = 0.9
reward = np.array([0, 1, 5])
transition = np.array([
    [0.5, 0.5, 0],
    [0, 0.5, 0.5],
    [0.5, 0, 0.5]
])
V = np.zeros(3)
for iteration in range(10):
    newV = reward + gamma *
    transition.dot(V)
    V = newV
    print("Iteration", iteration+1)
    print(V)
print("\nFinal State Values")
print(V)
```

OUTPUT:

```
=====
Iteration 1
[0. 1. 5.]
Iteration 2
[0.45 3.7 7.25]
Iteration 3
[1.8675 5.9275 8.465 ]
Iteration 4
[3.50775 7.476625 9.649625]
Iteration 5
[ 4.94296875  8.7068125 10.92081875]
Iteration 6
[ 6.14240156  9.83243406 12.13870437]
Iteration 7
[ 7.18867603 10.8870123 13.22649767]
Iteration 8
[ 8.13405975 11.85107949 14.18682817]
Iteration 9
[ 8.99331266 12.71705844 15.04439956]
Iteration 10
[ 9.76966699 13.4926561 15.8169705 ]

Final State Values
[ 9.76966699 13.4926561 15.8169705 ]

RL EXP5.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP5.py
File Edit Format Run Options Window Help
import numpy as np
gamma = 0.9
reward = np.array([0, 1, 5])
transition = np.array([
    [0.5,0.5,0],
    [0,0.5,0.5],
    [0.5,0,0.5]
])
V = np.zeros(3)

for iteration in range(10):

    newV = reward + gamma * transition.dot(V)

    V = newV

    print("Iteration", iteration+1)
    print(V)

print("\nFinal State Values")
print(V)
Ln: 8 Col: 2
```

EXPERIMENT 6: EXPLORATION VS. EXPLOITATION STRATEGIES

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

PROGRAM:

```
import random
epsilon = 0.2
Q = [10, 15, 8]
for episode in range(20):
    if random.random() < epsilon:
        action = random.randint(0,2)
        print("Explore -> Action", action)
    else:
        action = Q.index(max(Q))
        print("Exploit -> Action", action)
```

```
=====
Exploit -> Action 1
Exploit -> Action 1
Exploit -> Action 1
Exploit -> Action 1
Exploit -> Action 1
Explore -> Action 2
Exploit -> Action 1
Exploit -> Action 1
Exploit -> Action 1
Explore -> Action 0
Exploit -> Action 1
Exploit -> Action 1
Explore -> Action 1
Exploit -> Action 1
Explore -> Action 1
Exploit -> Action 1
Explore -> Action 0
Exploit -> Action 1
Exploit -> Action 1
Exploit -> Action 1

RL EXP6.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP6.py (3...
File Edit Format Run Options Window Help
import random
epsilon = 0.2
Q = [10, 15, 8]
for episode in range(20):
    if random.random() < epsilon:

        action = random.randint(0,2)

        print("Explore -> Action", action)

    else:

        | action = Q.index(max(Q))

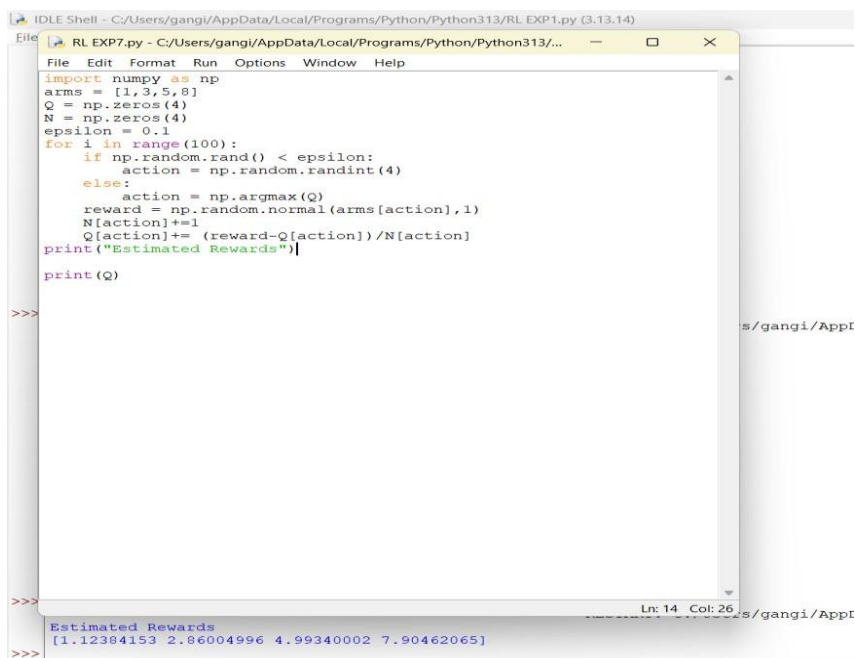
        print("Exploit -> Action", action)
Ln: 13 Col: 5
```

EXPERIMENT 7: MULTI-ARMED BANDIT PROBLEM

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action selection methods

PROGRAM:

```
import numpy as np
arms = [1,3,5,8]
Q = np.zeros(4)
N = np.zeros(4)
epsilon = 0.1
for i in range(100):
    if np.random.rand() < epsilon:
        action = np.random.randint(4)
    else:
        action = np.argmax(Q)
    reward = np.random.normal(arms[action],1)
    N[action]+=1
    Q[action] += (reward-Q[action])/N[action]
    print("Estimated Rewards")
    print(Q)
```



```
IDLE Shell - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP1.py (3.13.14)
File Edit Format Run Options Window Help
RL EXP7.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/...
import numpy as np
arms = [1,3,5,8]
Q = np.zeros(4)
N = np.zeros(4)
epsilon = 0.1
for i in range(100):
    if np.random.rand() < epsilon:
        action = np.random.randint(4)
    else:
        action = np.argmax(Q)
    reward = np.random.normal(arms[action],1)
    N[action]+=1
    Q[action] += (reward-Q[action])/N[action]
    print("Estimated Rewards")
    print(Q)
>>>
Estimated Rewards
[1.12384153 2.86004996 4.99340002 7.90462065]
>>>
```

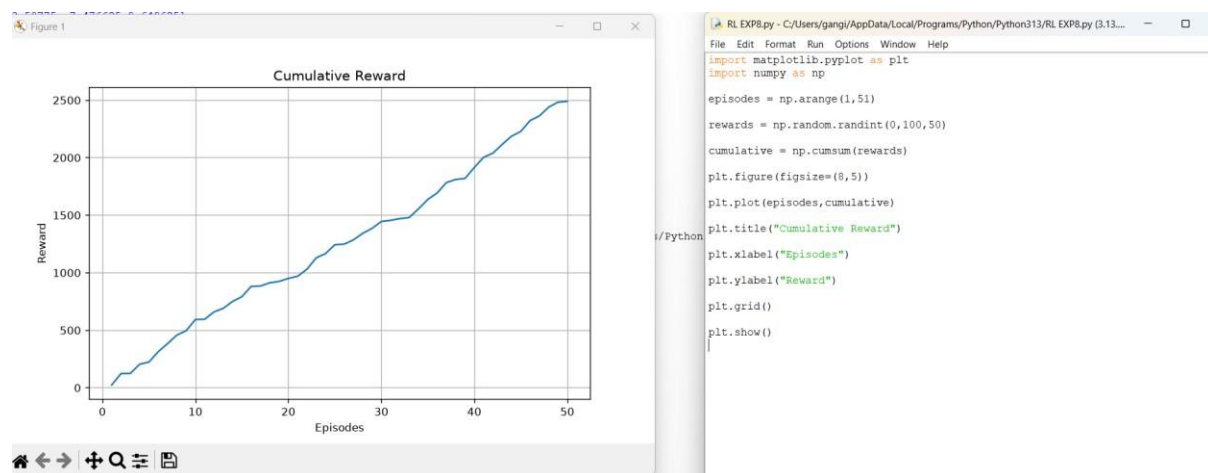
EXPERIMENT 8: REWARD VISUALIZATION IN REINFORCEMENT LEARNING

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

PROGRAM:

```
import matplotlib.pyplot as plt
import numpy as np
episodes = np.arange(1,51)
rewards = np.random.randint(0,100,50)
cumulative = np.cumsum(rewards)
plt.figure(figsize=(8,5))
plt.plot(episodes,cumulative)
plt.title("Cumulative Reward")
plt.xlabel("Episodes")
plt.ylabel("Reward")
plt.grid()
plt.show()
```

OUTPUT



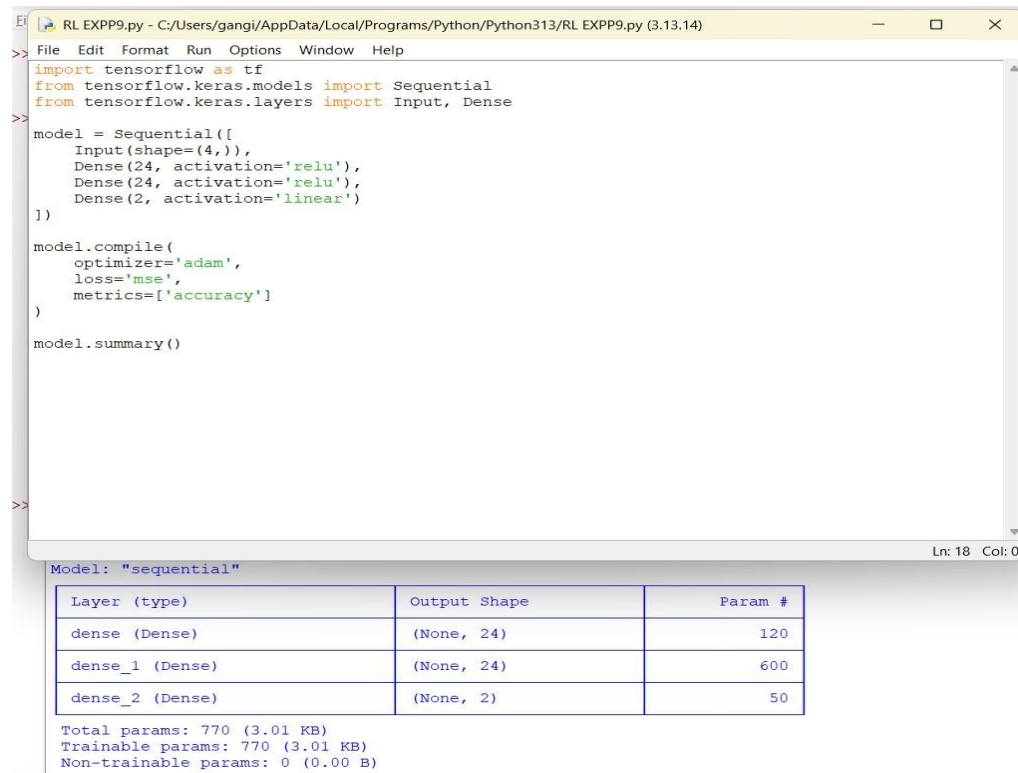
EXPERIMENT 9: TENSORFLOW AND KERAS-BASED NEURAL NETWORK FOR RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(24,input_shape=(4,),activation='relu'))
model.add(Dense(24,activation='relu'))
model.add(Dense(2,activation='linear'))
model.compile(optimizer='adam',
              loss='mse',
              metrics=['accuracy'])
model.summary()
```

OUTPUT



```
RL EXPP9.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXPP9.py (3.13.14)
File Edit Format Run Options Window Help
>> import tensorflow as tf
>> from tensorflow.keras.models import Sequential
>> from tensorflow.keras.layers import Input, Dense
>> model = Sequential([
>>     Input(shape=(4,)),
>>     Dense(24, activation='relu'),
>>     Dense(24, activation='relu'),
>>     Dense(2, activation='linear')
>> ])
>> model.compile(
>>     optimizer='adam',
>>     loss='mse',
>>     metrics=['accuracy']
>> )
>> model.summary()
>>
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 24)	120
dense_1 (Dense)	(None, 24)	600
dense_2 (Dense)	(None, 2)	50

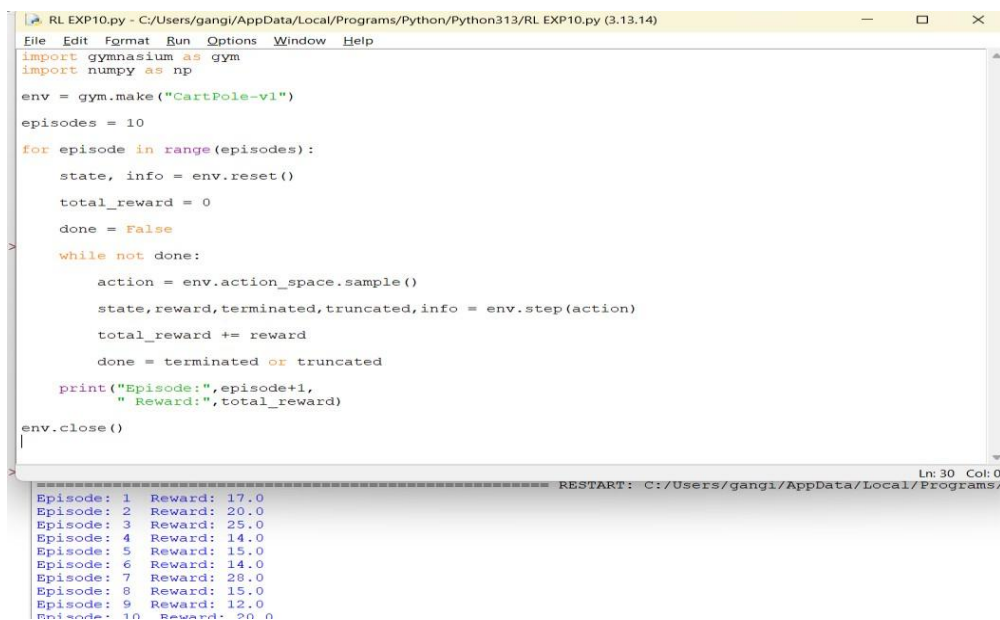
Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)

EXPERIMENT 10: MINI REINFORCEMENT LEARNING PROJECT USING CARTPOLE

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

PROGRAM:

```
import gymnasium as gym
import numpy as np
env = gym.make("CartPole-v1")
episodes = 10
for episode in range(episodes):
    state, info = env.reset()
    total_reward = 0
    done = False
    while not done:
        action = env.action_space.sample()
        state, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
        done = terminated or truncated
    print("Episode:", episode+1, " Reward:", total_reward)
env.close()
```



The screenshot shows a Jupyter Notebook window titled "RL EXP10.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP10.py (3.13.14)". The code in the notebook is identical to the program shown above. The output at the bottom of the notebook shows the results of 10 episodes:

```
Episode: 1 Reward: 17.0
Episode: 2 Reward: 20.0
Episode: 3 Reward: 25.0
Episode: 4 Reward: 14.0
Episode: 5 Reward: 15.0
Episode: 6 Reward: 14.0
Episode: 7 Reward: 28.0
Episode: 8 Reward: 15.0
Episode: 9 Reward: 12.0
Episode: 10 Reward: 20.0
```